# The MLOps Practitioner course

## Course guide

STRATEGY · TRACKS · CHECKLISTS · RUBRIC · TIMELINE

Aya Nasser Salama · MLOps MENA Community

CONTENTS

# What is in this guide

COURSE TIMELINE

# Six weeks, week by week

Sundays · 7:00 PM Cairo

Every live session is on a Sunday at 7:00 PM Cairo time. A mini project closes every session.

**SUN · AUG 16**
**Week 1 · From Notebook to Production-Ready Code**
Packaging, ML APIs, Docker, logging and testing — 7 lessons

**SUN · AUG 23**
**Week 2 · MLOps Core — Tracking, Versioning, Automation**
MLflow, DVC, GitHub Actions and Terraform — 6 lessons

**NO SESSION**
**Week 3 · Project Week — no lecture**
Build time: apply Weeks 1–2 and revise the first two modules

**SUN · SEP 6**
**Week 4 · Inference, Serving and Release Strategies**
Airflow, serving stacks, load testing and rollouts — 11 lessons

**SUN · SEP 13**
**Week 5 · Model Optimization — Faster, Smaller, Cheaper**
Pruning, quantization, distillation and compilation — 11 lessons

**SUN · SEP 20**
**Week 6 · Observability and Drift Detection**
Drift methods, dashboards, LLM tracing and guardrails — 12 lessons

# Foundations and automation

WEEK 1 · AUG 15–21

### From Notebook to Production-Ready Code

Live · Sun Aug 16, 7:00 PM GMT+3

- MLOps maturity model
- Python packaging and project structure
- Building ML APIs and serialization formats
- Docker and containerization
- Structured logging and testing with pytest

Mini project at the end of the session

WEEK 2 · AUG 22–28

### MLOps Core — Tracking, Versioning, Automation

Live session · Week 2

- MLflow experiment tracking
- Model registry and continuous training
- Data versioning with DVC
- CI/CD with GitHub Actions
- Infrastructure as code with Terraform

Mini project at the end of the session

WEEK 3 · AUG 29 – SEP 4

### Project Week — no lecture

No live session

- Start working on your chosen project
- Apply the principles from Weeks 1 and 2
- Implement and revise the first two modules

Catch-up and build time

# Serving, optimization, observability

**WEEK 4 · SEP 5–11**

### Inference, Serving and Release Strategies

Live session · Week 4

- Orchestration with Apache Airflow
- Three inference patterns: REST, batch, streaming
- Serving with FastAPI and BentoML
- GPU and CPU accelerated serving, LLM serving
- Load testing and release strategies

Mini project at the end of the session

**WEEK 5 · SEP 12–18**

### Model Optimization — Faster, Smaller, Cheaper

Live session · Week 5

- Pruning and quantization (PTQ, QAT)
- Knowledge distillation
- Compilation: TensorRT, ONNX Runtime, OpenVINO
- Edge deployment and LLM quantization
- Benchmarking and knowing when to stop

Mini project at the end of the session

**WEEK 6 · SEP 19–25**

### Observability and Drift Detection

Live session · Week 6

- Why models degrade: the drift taxonomy
- Statistical detection and Evidently AI
- Concept, label and embedding drift
- Prometheus, Grafana and Langfuse tracing
- RAG evaluation, cost monitoring, guardrails

Mini project at the end of the session

HOW TO GUARANTEE YOUR COMMITMENT

# Two certificates

Certifications · attendance · feedback · projects

ATTENDANCE CERTIFICATE

## Free — awarded to everyone who completes the course

- [ ] Attend all 5 sessions (or watch the recording)
- [ ] Fill the quick feedback form after EACH session
- [ ] Fill the final course feedback form at the end
- [ ] That’s it — no project required

PROJECT CERTIFICATE

## Optional — awarded for completing the full project

- [ ] Complete the end-to-end project (choose 1 of 2)
- [ ] Submit your GitHub repo + README
- [ ] Review at least 1 other student’s project
- [ ] Write a detailed written feedback (min. 300 words)

# Attendance, and how pairing works

### How I know you attended

After each session a short feedback form is shared in the group. It takes 2 minutes. Fill it → you're marked present. Skip it → that session doesn't count, even if you were in the room.

### Peer review — project certificate only

You review 1 project from another student. I assign the pairing. Your review must cover: code quality, README clarity, one thing done well, two things to improve, and how you would extend the project.

*Two project tracks — choose one. Both graded on the same 10-point rubric.*

# What to have in place first

You will get far more out of the five sessions if you arrive with these two things done.

1

## Study the fundamentals

At minimum, the supervised learning course from Andrew Ng's Machine Learning Specialization on Coursera. You need to understand what a model is before you learn how to ship one.

coursera.org → Machine Learning Specialization

2

## Train one model yourself

Do at least one project on Kaggle that trains a simple ML model — any data type: video, images, CSVs, audio or text. The subject does not matter. Having trained something end to end does.

*Both are free. Do them before Session 1, not during it.*

CHOOSE YOUR TRACK

# Two projects, one standard

PROJECT 1 — DEEP LEARNING

## Arabic Sentiment Analysis

PyTorch · AraBERT · TensorRT · BentoML · Triton

Best if you have a GPU and are interested in model optimization

PROJECT 2 — LLM / RAG

## Arabic Legal Document Q&A

vLLM · RAG · RAGAS · Langfuse · Guardrails

Best if no GPU — focused on evaluation and observability

*Both tracks · same 10-point rubric · same deadline · same certificate*

STRATEGY

# The project strategy

One project. Five sessions. One working piece per session.

S1

**Package + API**

Containerized FastAPI /predict running locally

→

S2

**Track + Automate**

MLflow · DVC · GitHub Actions on every push

→

S3

**Serve + Release**

BentoML/vLLM · Locust tested · Canary config

→

S4

**Monitor**

Drift detection · Grafana dashboard · Alerts

→

S5

**Optimize**

Quantized model · Benchmark table + README

FOUR RULES

# How the project runs

### 1 Choose once

DL or LLM — commit at Session 1 and build the same project all 5 sessions.

### 2 Commit per session

At least one GitHub commit per session. Code is your evidence of learning.

### 3 Same rubric

10 identical criteria. No track is easier than the other.

### 4 Peer review is required

You cannot earn the project cert without reviewing another student.

# The 10-point rubric

Same criteria for both tracks — no track is easier. Criteria 01–05.

| # | AREA | SESSION | WHAT YOU BUILD | PASS CONDITION |
|---|---|---|---|---|
| 01 | **Code & Packaging** | S1 | pyproject.toml, src/ layout, OOP model class, type hints | pip install -e . works. Model loads without errors. |
| 02 | **API Endpoint** | S1 | /predict with Pydantic, /health, async handlers | curl /predict returns correct output. |
| 03 | **Docker** | S1 | Dockerfile, docker-compose.yml, 3-command README | docker compose up && curl localhost:8000/predict works on reviewer's machine. |
| 04 | **MLflow Tracking** | S2 | ≥ 5 runs, params + metrics + artifacts, best model in Registry | MLflow UI shows comparison. Model promoted to Production. |
| 05 | **DVC Data Versioning** | S2 | Dataset tracked, dvc.yaml pipeline, dvc repro works | git checkout + dvc pull + dvc repro reproduces result. |

RUBRIC · BOTH TRACKS

# The 10-point rubric

Criteria 06–10.

| # | AREA | SESSION | WHAT YOU BUILD | PASS CONDITION |
|---|---|---|---|---|
| 06 | **GitHub Actions CI/CD** | S2 | Lint → test → build → push on every PR. Quality gate. | Merge a PR. Actions green. Docker image pushed to registry. |
| 07 | **Production Serving** | S3 | BentoML (DL) or vLLM + BentoML (LLM). Locust tested. | p95 latency documented. Locust report in /reports/. |
| 08 | **Monitoring** | S4 | ≥ 1 drift metric (DL) OR RAGAS faithfulness (LLM). Grafana panel. | Screenshot in README. Alert threshold set. |
| 09 | **Peer Review** | Post S5 | Written review ≥ 300 words, all 5 areas | Submitted on time. Covers: setup, code, strength, 2 improvements, extension. |
| 10 | **README & Architecture** | S5 | 3-command setup, architecture diagram, session changelog | Reviewer runs it without asking anything. |

PROJECT 1 · DEEP LEARNING TRACK

# Arabic Sentiment Analysis

E-commerce product reviews in Arabic · PyTorch · AraBERT · BentoML · TensorRT · Triton

## The business problem

An Arab e-commerce platform (Noon.com, Amazon.eg) receives 50,000 Arabic product reviews per day. They need to classify each as Positive, Negative, or Neutral — in Arabic — to power product rankings, seller ratings, and customer service prioritization.

## Why this model for this course

— GPU optimization and TensorRT actually matter here

— Arabic NLP — directly relevant to MENA students

— Real drift — Ramadan vocabulary, slang, dialect shifts

— All 3 inference patterns apply: web, batch, streaming

— Session 5 distillation: 12-layer → 6-layer student

# What you ship per session

| | | |
|---|---|---|
| S1 | **Package + API** | AraBERT wrapper, /predict: {"text":"المنتج رائع"} → {"label":"positive","confidence":0.94} |
| S2 | **Track + Version** | MLflow: 5+ runs. DVC: labeled reviews. GitHub Actions CI on push. |
| S3 | **Serve** | BentoML micro-batching. TensorRT FP16. Locust: p95 before vs after. |
| S4 | **Monitor** | Evidently PSI on vocabulary. Prometheus + Grafana. Alert: PSI > 0.25. |
| S5 | **Optimize** | Distil 12→6 layers. INT8 ONNX. Benchmark: accuracy × latency × size. |

# Completion checklist

Code, packaging, tracking and versioning. Verified together with the instructor. Part 1 of 2.

## CODE & PACKAGING

- [ ] src/ layout with pyproject.toml — pip install -e . works
- [ ] AraBERT or CAMeL-BERT loaded in a class with type hints
- [ ] FastAPI /predict: {text: str} → {label, confidence, model_version}
- [ ] Pydantic rejects empty text — 422 returned, tested with curl
- [ ] /health returns {status: healthy} — used by Docker healthcheck
- [ ] docker build -t arabic-sentiment . passes
- [ ] docker compose up starts service on port 8000
- [ ] README: exactly 3 commands to run on any machine

## EXPERIMENT TRACKING & DATA VERSIONING

- [ ] MLflow tracking server running — UI accessible
- [ ] ≥ 5 runs: model_name, lr, batch_size, accuracy, f1_macro
- [ ] MLflow comparison screenshot in /reports/mlflow_comparison.png
- [ ] Best model registered as ‘ArabicSentiment’ → promoted to Production

- [ ] Labeled dataset tracked with DVC — dvc status shows clean
- [ ] dvc repro reproduces training with same metrics (± 0.005)
- [ ] GitHub Actions: lint → test → docker build → push on main
- [ ] CI fails if f1_macro drops below baseline — documented in README

# Completion checklist

Serving, monitoring and optimization. Part 2 of 2.

### SERVING & RELEASE

- [ ] BentoML service with batchable=True Runner — bentoml models list shows model
- [ ] bentoml serve runs /predict at localhost:8000
- [ ] Locust report before optimization in /reports/locust_fastapi.html
- [ ] TensorRT FP16 engine built — committed or documented
- [ ] Locust after TensorRT in /reports/locust_trt.html — p95 in README
- [ ] Batch scoring: ≥ 1,000 samples, output to /data/scoring/output/
- [ ] nginx canary config: weight=5/95 — rollout stages in README

### MONITORING & OBSERVABILITY

- [ ] Evidently DataDrift report on latest batch — saved in /reports/
- [ ] PSI computed for text_length and confidence_score features
- [ ] Drift check triggered after every batch scoring run
- [ ] Prometheus exposes PSI score at /metrics
- [ ] Grafana: ≥ 2 panels (p95 + PSI) — screenshot in README
- [ ] Alert: PSI > 0.25 → notification documented in README

### MODEL OPTIMIZATION

- [ ] 12-layer AraBERT distilled to 6-layer student — model file committed
- [ ] INT8 ONNX quantization applied — model_int8.onnx committed
- [ ] Benchmark table: original vs distilled vs INT8 (accuracy, p95, size)
- [ ] Final architecture diagram in README covering all 5 sessions

PROJECT 2 · LLM / RAG TRACK

# Arabic Legal Document Q&A

Arabic legal documents · Embeddings + Vector DB · vLLM · RAGAS · Langfuse · Guardrails

## The business problem

A law firm or government authority in the Arab world needs a system that answers questions about Arabic legal documents — contracts, Egyptian Civil Code articles, Saudi regulations — accurately and with citations. Hallucinations are legally unacceptable.

## Why this project for this course

— RAGAS evaluation is genuinely hard — deep S4 content

— No GPU required to start — API-based LLM works fine

— Embedding drift is real — legal language shifts across jurisdictions

— Langfuse tracing reveals the full RAG chain

— AWQ quantization in S5 makes the LLM run fully offline

# What you ship per session

| | | |
|---|---|---|
| S1 | **Package + API** | src/rag.py (ingest+query), /ask: {"question":"ما هي شروط العقد؟"} → {"answer":…, "sources":[…]} |
| S2 | **Track + Version** | MLflow: chunking experiments (chunk_size, overlap, RAGAS faithfulness). DVC: document corpus. |
| S3 | **Serve** | BentoML wraps RAG. vLLM serves generative model. Streaming /ask. |
| S4 | **Monitor** | RAGAS on ≥ 50 questions. Langfuse traces every request. Cosine drift. Token cost. |
| S5 | **Optimize** | AWQ-4bit generative model. Re-ranker distillation. RAGAS before vs after. |

# Completion checklist

Code, packaging, tracking and versioning. Verified together with the instructor. Part 1 of 2.

## CODE & PACKAGING

- [ ] src/ layout with pyproject.toml — pip install -e . works
- [ ] Ingestion pipeline: PDF/text → chunk → embed → vector store
- [ ] FastAPI /ask: {question: str} → {answer: str, sources: list[str]}
- [ ] Pydantic rejects empty question — 422 returned and tested
- [ ] /health returns {status: healthy, documents_indexed: N}
- [ ] Dockerfile includes vector store and embedded documents
- [ ] docker compose up starts service on port 8000
- [ ] README: 3 commands to run Q&A on any machine

## EXPERIMENT TRACKING & DATA VERSIONING

- [ ] MLflow logs each config: chunk_size, overlap, embedding_model, faithfulness
- [ ] ≥ 5 runs compared — MLflow screenshot in /reports/
- [ ] Best chunking config registered in MLflow Registry
- [ ] Document corpus tracked with DVC — dvc pull fetches all source docs
- [ ] dvc repro re-indexes corpus reproducibly from tracked documents
- [ ] GitHub Actions: lint → test → rebuild index → push Docker image
- [ ] CI fails if RAGAS faithfulness < 0.75 on 20-question test set

# Completion checklist

Serving, monitoring and optimization. Part 2 of 2.

### SERVING & RELEASE

- [ ] BentoML wraps RAG pipeline with async /ask endpoint
- [ ] vLLM serves the generative LLM — model name in README
- [ ] Streaming: tokens appear progressively in curl output
- [ ] Locust report at 50 concurrent users in /reports/
- [ ] Batch re-indexing script tested with at least one new document
- [ ] Canary rollout config documented in README

### MONITORING & OBSERVABILITY

- [ ] RAGAS on ≥ 50 questions — all 4 metrics logged
- [ ] RAGAS results stored in MLflow — trend visible across sessions
- [ ] Alert: faithfulness < 0.80 → notification triggered
- [ ] Langfuse self-hosted — every /ask creates a trace with spans
- [ ] RAGAS faithfulness score attached to each Langfuse trace
- [ ] Cosine similarity on query embeddings vs baseline — drift logged
- [ ] Token usage in Prometheus — Grafana shows cost/hour
- [ ] Guardrails PII detection active on all /ask responses

### MODEL OPTIMIZATION

- [ ] Generative model quantized to AWQ-4bit — weights documented
- [ ] RAGAS scores: original vs quantized — ≤ 0.03 faithfulness drop
- [ ] Latency before vs after quantization in README
- [ ] Final architecture diagram covering all 5 session components

# Peer review

Required for the project certificate — you cannot earn it without reviewing another student.

### AS THE REVIEWER — WHAT YOU SUBMIT

- [ ] Cloned the assigned student's repo and followed the README
- [ ] Service started — documented any issues encountered
- [ ] Written review ≥ 300 words as a GitHub Issue on their repo
- [ ] PDF copy of the review sent to the instructor
- [ ] Section 1 — Setup: did it run? how long? what broke?
- [ ] Section 2 — Code Quality: 2 specific patterns (good + refactor)
- [ ] Section 3 — One Genuine Strength: specific, production context
- [ ] Section 4 — Two Specific Improvements: concrete and actionable
- [ ] Section 5 — Extension Idea: what, why, how — one paragraph

### AS THE REVIEWEE — WHAT YOU RESPOND

- [ ] Read the review carefully before responding
- [ ] One GitHub comment per review section
- [ ] Each improvement: implement it or explain why you disagree
- [ ] Extension idea: acknowledge and note if you plan to implement it
- [ ] Response submitted within 3 days of receiving the review

*Peer review pairing is assigned by the instructor after Session 5.*

# How to write a good review

Minimum 300 words. All 5 areas. I read every review — quality matters.

| | | | |
|---|---|---|---|
| 1 | **Can I run it?** | Clone. Follow README. Does it work? How long? What broke? | Document every friction point. If it ran fine, note what was clear. |
| 2 | **Code Quality** | Type hints? OOP? Model loaded once? Repeated code? | Name 2 patterns: one that's good, one to refactor. Be specific. |
| 3 | **One Genuine Strength** | The ONE thing done notably well. 'Good code' is not useful. | One paragraph. What impressed you and why it matters in production. |
| 4 | **Two Improvements** | Not 'add more tests'. Specific enough to act on immediately. | Example: 'Locust wait_time should be between(0.5, 2.0) for realism' |
| 5 | **Extension Idea** | One concrete thing to build next to extend scope or production-readiness. | What it is, why it matters, how you'd approach it — one paragraph. |

*Submit as a GitHub Issue AND send a PDF to the instructor · Student responds with one comment per point*

# Timeline

During the course — one commit per session.

| | |
|---|---|
| **End of Session 1** | GitHub repo created. /predict running in Docker. 3-command README. |
| **End of Session 2** | ≥ 5 MLflow runs logged. DVC pipeline clean. GitHub Actions green. |
| **End of Session 3** | BentoML or vLLM service running. Locust report submitted. |
| **End of Session 4** | Drift detection or RAGAS running. Grafana dashboard live. |
| **End of Session 5** | Optimization complete. Benchmark table. Start final polish. |

**2**

**weeks after Session 5**

to submit everything

*2 weeks after Session 5 — submit your GitHub repo AND your peer review together. No separate deadlines.*

# Referral to partner companies

Top projects are referred to our partner companies — the same referral route already offered to community moderators.

## HOW YOU QUALIFY

- Complete the end-to-end project and earn the project certificate
- Score highly on the 10-point rubric — same criteria for both tracks
- Ship a repository a reviewer can run without asking a question
- Complete your peer review on time

Attendance certificate alone does not qualify

## WHAT YOU GET

**Your project put in front of hiring partners, by name.**

Referrals go out after Session 5, once peer reviews are in.

At the end

# What I need from you!